

UNIVERSITÉ PARIS 1 PANTHÉON-SORBONNE
MASTER 1 MIAGE — ATELIER IA

Analyse du marché de l'emploi Data en Île-de-France

*Collecte automatisée via l'API France Travail,
nettoyage, calcul et analyse*

Question de recherche

*« Quelles compétences Data sont les plus demandées en Île-de-France et comment
se valorisent-elles salarialement ? »*

Yanis Hamitouche • Mehdi Atmane

29 juin 2026

Table des matières

1	Introduction et contexte	2
2	Plan de récolte de données	2
2.1	Comparaison des techniques de collecte	2
2.2	Présentation de la source	2
2.3	Authentification OAuth2	3
2.4	Définition du périmètre	3
2.5	Pagination et respect du quota	3
2.6	Volumétrie obtenue	3
3	Stratégie de nettoyage	3
3.1	Dédoublonnage	4
3.2	Normalisation des champs	4
3.3	Parsing du salaire	4
3.4	Détection des compétences	4
3.5	Bilan de qualité des données	4
4	Calcul des indicateurs	4
5	Analyse	5
5.1	Panorama du corpus	5
5.2	Compétences les plus demandées	5
5.3	Valorisation salariale par compétence	6
5.4	Synthèse : demande croisée à la valorisation	6
6	Conclusion	6
6.1	Réponse à la question fil rouge	6
6.2	Limites et biais	6
6.3	Pistes d'amélioration	7
A	Reproductibilité	7

1 Introduction et contexte

Ce dossier présente une analyse du marché de l'emploi dans les métiers de la donnée (Data Analyst, Data Engineer, Data Scientist, Business Intelligence) en Île-de-France, construite à partir de données collectées *automatiquement*. L'objectif est double : (i) démontrer une chaîne de traitement reproductible — de la collecte à l'analyse — et (ii) répondre, chiffres à l'appui, à la question fil rouge énoncée en page de titre.

La démarche suit cinq étapes, matérialisées par des scripts numérotés et ré-exécutables : collecte (01_collecte.py), nettoyage (02_nettoyage.py), calcul (03_calculs.py), analyse (04_analyse.ipynb) et synthèse (le présent dossier). Chaque choix technique est justifié ci-dessous.

2 Plan de récolte de données

2.1 Comparaison des techniques de collecte

Trois familles de techniques, vues en cours, permettent de récolter des offres d'emploi en ligne. Nous les avons évaluées selon quatre critères déterminants pour un travail rigoureux et reproductible.

Critère	API REST (France Travail)	Scraping job board
Légalité	Licence de réutilisation officielle	Souvent contraire aux CGU
Robustesse	JSON structuré et versionné	HTML fragile, casse aux re-fontes
Pertinence	Aucun rendu JS à charger	Navigateur <i>headless</i> lourd
Reproductibilité	Requêtes paramétrées re-jouables	Dépend du DOM courant

Table 1 – Comparaison des approches de collecte.

Conformément au principe « **API si disponible** > **parsing HTML** > **navigateur headless** », nous avons retenu l'**API REST officielle « Offres d'emploi v2 » de France Travail** (ex-Pôle Emploi). Le *scraping* d'Indeed, LinkedIn ou Welcome to the Jungle a été écarté : il est juridiquement risqué (CGU), techniquement fragile (le HTML change sans préavis) et superflu puisqu'une API officielle expose exactement la donnée recherchée. Selenium / Playwright n'auraient été qu'un dernier recours en l'absence d'API ; ici aucun rendu JavaScript n'est à charger, donc aucun navigateur *headless* n'est nécessaire.

2.2 Présentation de la source

L'API expose les offres diffusées par France Travail et ses partenaires via une ressource de recherche paginée :

GET <https://api.francetravail.io/partenaire/offresemploi/v2/offres/search>

Chaque offre est un objet JSON riche : identifiant, intitulé, description, dates, lieu de travail (commune, département, coordonnées), entreprise, type de contrat, expérience, salaire, compétences, code et libellé ROME, secteur d'activité. Cette structure stable nous dispense de tout *parsing* HTML.

2.3 Authentification OAuth2

L'accès repose sur **OAuth2** avec le **grant client_credentials** : une authentification *applicative* (sans utilisateur final), adaptée à un traitement automatisé. Le script échange l'identifiant et le secret client contre un jeton *Bearer*.

```
def obtenir_token(config):
    reponse = requests.post(config["token_url"], data={
        "grant_type": "client_credentials",
        "client_id": config["client_id"],
        "client_secret": config["client_secret"],
        "scope": config["scope"],
    })
    reponse.raise_for_status()
    return reponse.json()["access_token"]
```

Deux valeurs ont été **vérifiées sur la documentation Swagger** plutôt qu'inventées :

- **Endpoint token** : `https://entreprise.francetravail.fr/connexion/oauth2/access_token?realm=`
- **Scopes obligatoires** : `api_offresdemploiv2 o2dsoffre`

Sécurité des secrets. Les identifiants ne figurent jamais en clair : ils résident dans un fichier `.env` exclu du dépôt (`.gitignore`), tandis qu'un `.env.example` documente les clés attendues. C'est une bonne pratique essentielle pour un dépôt partagé en binôme.

2.4 Définition du périmètre

- **Géographie** : Île-de-France via le paramètre `region=11`. Ce choix a été préféré au filtre `departement`, *limité à 5 valeurs par l'API* alors que l'IDF compte 8 départements. Le département reste récupérable dans chaque offre (champ `lieuTravail`) pour l'analyse fine.
- **Métiers** : recherche par mots-clés — `data analyst`, `data engineer`, `data scientist`, `business intelligence`, `BI` — avec un fichier brut distinct par mot-clé pour tracer l'origine de chaque offre.
- **Fenêtre temporelle** : offres créées sur les **30 derniers jours** (`minCreationDate / maxCreationDate`), soit une photographie récente du marché.

2.5 Pagination et respect du quota

L'API renvoie au plus **150 offres par appel** via le paramètre `range=p-d`, avec les bornes imposées $p \leq 3000$ et $d \leq 3149$. Le script parcourt les pages tant que le code HTTP vaut 206 (réponse partielle) et s'arrête sur 200 (page complète) ou 204 (aucune offre). Une pause de 0,3 s entre les appels respecte le quota d'environ **4 appels/seconde** par application ; les erreurs transitoires (429, 5xx) déclenchent des relances avec *back-off*.

2.6 Volumétrie obtenue

La collecte a produit **331 offres brutes** (avant dédoublonnage), écrites en JSON UTF-8 dans `data/raw/` — dossier traité comme **immuable** et non versionné. Aucun mot-clé n'a atteint le plafond de l'API : la photographie du marché sur la fenêtre considérée est donc complète.

3 Stratégie de nettoyage

Le script `02_nettoyage.py` transforme les JSON bruts en tables propres (`data/processed/`). Principe directeur : `data/raw/` n'est *jamais* modifié ; tout traitement est rejouable à l'identique.

3.1 Dédoublonnage

Une même offre peut remonter sur plusieurs mots-clés (par exemple un poste « Data Engineer (BI) »). Le dédoublonnage par identifiant `id` ramène le corpus de **331 à 299 offres uniques** (−32 doublons), évitant de sur-pondérer les offres polyvalentes.

3.2 Normalisation des champs

Les objets imbriqués sont aplatis en colonnes exploitables. En particulier, le **département** est dérivé du préfixe « MN - VILLE » du champ `lieuTravail.libelle` ; l'entreprise, le type de contrat, l'expérience, le métier ROME et le secteur sont extraits dans des colonnes dédiées, et les dates sont typées.

3.3 Parsing du salaire

Le salaire est fourni en **texte libre** (`salaire.libelle`) et n'est présent que sur une minorité d'offres. Une expression régulière extrait la période, le minimum et le maximum des formats observés (par exemple « *Annuel de 50000.0 Euros à 55000.0 Euros sur 12 mois* »), puis **normalise en base annuelle** selon les hypothèses suivantes, explicitées car elles introduisent une approximation :

Période	Facteur annuel	Hypothèse
Annuel	$\times 1$	valeur déjà annuelle
Mensuel	$\times N$	N versements/an (12 par défaut)
Horaire	$\times 1820$	temps plein 35 h $\times$ 52 semaines

Table 2 – Normalisation des salaires en base annuelle.

L'hypothèse horaire est la plus fragile (sensible au temps partiel) ; elle est donc signalée. Au final, **27 % des offres** disposent d'un salaire annuel exploitable : c'est une limite assumée, quantifiée et discutée en conclusion.

3.4 Détection des compétences

Le champ structuré `competences[]` n'est renseigné que sur **~15 %** des offres — insuffisant pour répondre à une question portant précisément sur les compétences. Nous avons donc construit un **lexique curé de technologies Data** (langages, bases de données, big data, cloud, BI, *machine learning*, outils) et détecté leur présence par expressions régulières dans l'intitulé, la description et les libellés de compétences. La couverture passe ainsi à **75 % des offres**.

Justification et limites. Ce choix méthodologique est un compromis explicite entre exhaustivité et précision. Les motifs courts ou ambigus (par exemple **SAS**, qui désigne aussi une forme juridique, ou **R**, lettre isolée) sont volontairement encadrés par des frontières de mots ; un contrôle manuel estime les faux positifs résiduels à *environ 5 %* sur ces motifs — acceptable au regard du gain de couverture (de 15 % à 75 %). Le résultat est matérialisé par une table au format long (`id`, `competence`) qui facilite les comptages en aval.

3.5 Bilan de qualité des données

4 Calcul des indicateurs

Le script `03_calculs.py` produit les agrégats consommés par le notebook, de sorte que l'analyse ne recalcule rien de lourd :

- `top_competences.csv` : volume et part des offres par compétence ;

Indicateur	Valeur
Offres brutes collectées	331
Offres uniques (après dédoublonnage)	299
Départements distincts couverts	8
Entreprises distinctes	97
Compétences distinctes détectées	53
Taux de salaire annuel exploitable	27 %
Couverture compétences (détection texte)	75 %

Table 3 – Synthèse de la qualité du corpus nettoyé.

- `competences_salaire.csv` : salaire annuel médian et quartiles par compétence, **filtré sur un effectif ≥ 5** pour écarter les médianes peu fiables ;
- répartitions par département, type de contrat, expérience et métier ROME ;
- `kpi_global.csv` : indicateurs de synthèse.

Choix de la médiane. Les salaires sont résumés par la **médiane** plutôt que la moyenne, afin de résister aux valeurs extrêmes — un salaire aberrant à 492 000 € est en effet présent dans les données. Le seuil d'effectif (≥ 5) garantit que les comparaisons inter-compétences reposent sur une base statistique minimale. **Chiffres clés** : salaire annuel médian $\approx 50\,000$ € (Q1 ≈ 37 k€, Q3 ≈ 56 k€), 69 % de CDI.

5 Analyse

L'analyse complète et interactive figure dans `notebooks/04_analyse.ipynb` (ré-exécutable de bout en bout). Les figures ci-dessous en synthétisent les résultats.

5.1 Panorama du corpus

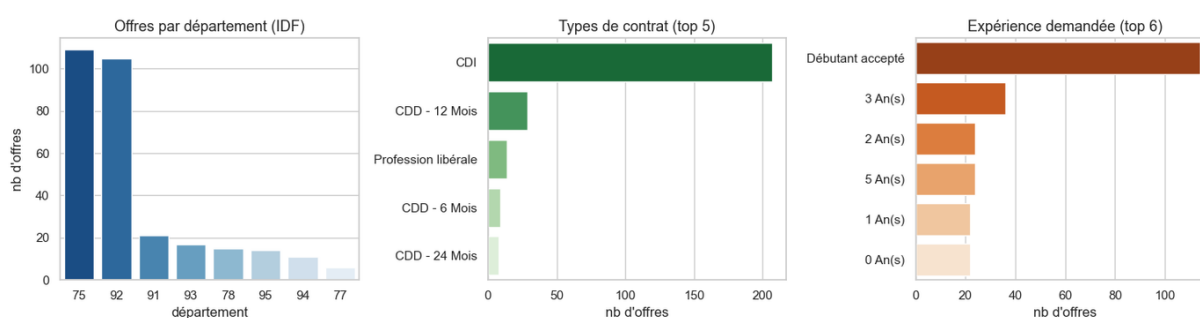


Figure 1 – Répartition des offres par département, type de contrat et expérience.

L'emploi Data francilien est **fortement concentré sur Paris (75) et les Hauts-de-Seine (92)**, qui regroupent l'essentiel des offres — reflet de la localisation des sièges sociaux et des ESN. Le **CDI domine** nettement, et les offres ciblent surtout des profils expérimentés.

5.2 Compétences les plus demandées

SQL et **Python** constituent le socle incontournable, présents dans une large part des offres. Suivent la **BI / dataviz** (Power BI, Excel, Tableau) et l'écosystème **cloud / big data** (Azure, GCP, AWS, Spark, Databricks). La demande se structure ainsi en deux familles : un noyau analytique et une couche d'ingénierie de la donnée.

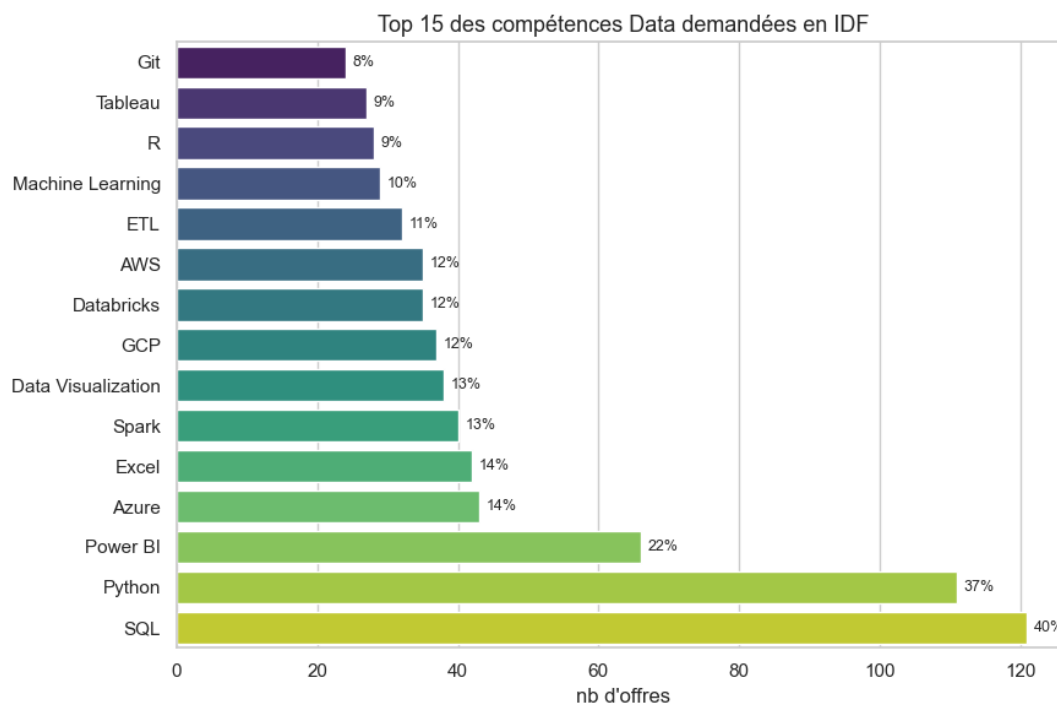


Figure 2 – Top 15 des compétences Data demandées (part des offres en étiquette).

5.3 Valorisation salariale par compétence

Les compétences les mieux rémunérées relèvent du **cloud et du big data** (GCP, AWS, Azure, Spark, Kafka, Databricks, Snowflake), au-dessus du socle analytique. La maîtrise de l'*infrastructure* de données se valorise donc davantage que les seuls outils de restitution (BI).

5.4 Synthèse : demande croisée à la valorisation

Ce croisement distingue deux profils de compétences : des compétences *socle*, très demandées mais peu différenciantes salarialement (SQL, Python), et des compétences *premium* (cloud, traitement distribué) qui tirent les rémunérations vers le haut tout en étant moins universelles.

6 Conclusion

6.1 Réponse à la question fil rouge

En Île-de-France, le marché Data repose sur un **socle SQL + Python** quasi universel, complété par la **BI** et, de plus en plus, par l'**ingénierie de la donnée dans le cloud**. Côté rémunération, **ce sont les compétences cloud et big data qui se valorisent le mieux** : SQL et Python sont nécessaires mais peu distinctifs, tandis qu'un profil *Data Engineer* maîtrisant un cloud (Azure / GCP / AWS) et le traitement distribué (Spark / Databricks) se situe en haut de l'échelle salariale, devant le *Data Analyst* BI seul.

6.2 Limites et biais

- **Biais de source** : les offres France Travail ne représentent *pas* le marché total. De nombreux postes cadres sont diffusés via d'autres canaux (LinkedIn, cabinets, cooptation) ; le haut du marché est probablement sous-estimé.
- **Couverture salariale** : seules 27 % des offres affichent un salaire exploitable — les médianes par compétence, fondées sur de petits effectifs (d'où le seuil ≥ 5), sont à lire comme des

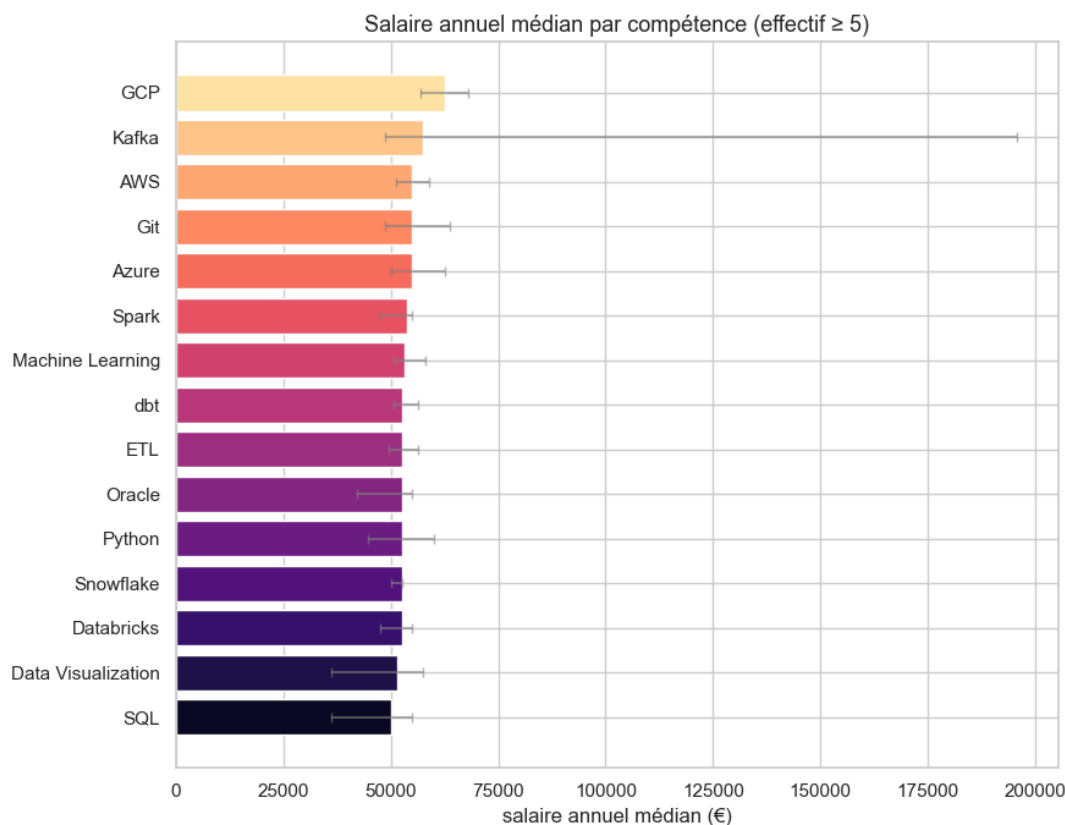


Figure 3 – Salaire annuel médian par compétence (effectif ≥ 5 ; barres d'erreur = intervalle interquartile).

ordres de grandeur.

- **Détection lexicale** : quelques faux positifs résiduels ($\sim 5\%$ sur les motifs courts), et un lexique curé mais non exhaustif.
- **Fenêtre temporelle** : une collecte sur 30 jours est une photographie, non une tendance ; elle peut refléter une saisonnalité.

6.3 Pistes d'amélioration

Élargir la fenêtre temporelle et croiser plusieurs sources (autres API, données INSEE / DARES) ; enrichir le lexique et fiabiliser la détection par des techniques de TAL (NLP) ; suivre l'évolution dans le temps pour passer d'une photographie à une véritable analyse de tendance.

A Reproductibilité

Le dépôt est organisé selon une structure standard (`src/`, `data/raw`, `data/processed`, `notebooks/`, `rapport/`). Après installation des dépendances (`pip install -r requirements.txt`) et configuration du fichier `.env`, le pipeline s'exécute dans l'ordre :

```
python src/01_collecte.py # collecte API -> data/raw/
python src/02_nettoyage.py # nettoyage -> data/processed/
python src/03_calculs.py # KPI / agrégats
jupyter notebook notebooks/04_analyse.ipynb
```

Les figures de ce dossier sont régénérées par `python rapport/build_figures.py`.

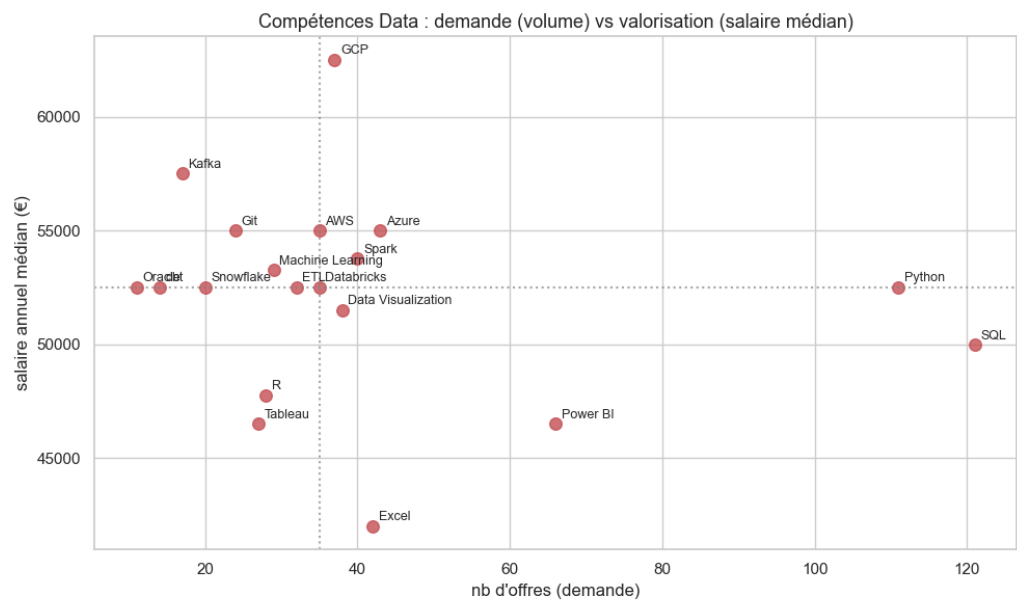


Figure 4 – Positionnement des compétences selon la demande (axe X) et la valorisation salariale médiane (axe Y).